RESOLUÇÃO COMENTADA

Dojo Digital

Do repositório sem deploy até a plataforma no ar, com o porquê de cada decisão.

01 Antes de começar

Três coisas resolvidas agora evitam problema depois. Nenhuma delas aparece no enunciado, e todas as três derrubam grupo.

1 **Escolha a região e não mude mais**

Este guia usa **us-east-1** (Norte da Virgínia), porque é a região mais barata e a mais bem documentada. São Paulo (`sa-east-1`) tem latência menor para Porto Alegre, mas custa mais: a comparação está na seção 07.

Recursos não se enxergam entre regiões. Se a instância nascer em uma e o bucket em outra, nada funciona e o erro não é óbvio.

2 **Crie um alarme de orçamento**

No **AWS Budgets**, crie um orçamento de custo mensal de US$ 5 com alerta por e-mail em 80%. Custa zero e é a única proteção real contra a instância esquecida ligada.

3 **Gere o par de chaves SSH**

Em **EC2 → Key pairs → Create key pair**, tipo RSA, formato `.pem`. O arquivo é baixado **uma única vez**: se perder, não tem como recuperar, só criar outro par e outra instância.

```
chmod 400 ~/Downloads/dojo.pem
```

> **Não confunda os dois pares de chaves deste desafio.** O par de chaves **SSH** é o que abre a instância e é assunto da tarefa 01. O par de chaves de **acesso** (Access Key ID e Secret Access Key) é o que o backend usa para falar com o S3, e é assunto da tarefa 04. São coisas diferentes, com nomes parecidos.

02 A instância EC2 e o Security Group

1 Suba a instância

EC2 → Launch instance. Ubuntu Server LTS, tipo `t3.micro`, o par de chaves criado acima, e **Auto-assign public IP** habilitado. Disco: os 8 GB padrão bastam.

2 Configure o Security Group

É o firewall da instância, e é onde a maior parte dos pontos da tarefa 01 está. Só três regras de entrada:

PORTA	ORIGEM	POR QUE ESSA ORIGEM
`22` SSH	`SEU_IP/32`	Só você administra a máquina. `0.0.0.0/0` aqui expõe o SSH ao mundo inteiro e é reprovação automática.
`3000` API	`0.0.0.0/0`	O navegador de qualquer aluno precisa alcançar a API, e não dá para prever de qual IP. Aqui o mundo inteiro é a resposta certa.
`5432`	—	**Não crie esta regra.** Quem fala com o banco é a API, e ela está dentro da mesma instância.

> O Security Group tem **estado**: você libera a entrada e a resposta sai sozinha. Não é preciso criar regra de saída para o tráfego de resposta.

3 Conecte e instale o Docker

```
ssh -i ~/Downloads/dojo.pem ubuntu@IP_PUBLICO

sudo apt-get update
sudo apt-get install -y docker.io docker-compose-v2 git

# libera o docker sem sudo
sudo usermod -aG docker ubuntu
newgrp docker
```

O `newgrp` vale só para o terminal atual. Se em outro terminal aparecer `permission denied ... docker.sock`, saia do SSH e entre de novo.

4 Clone o repositório

```
git clone https://github.com/Liga-de-Computacao-em-Nuvem-PUCRS/dojo-digital.git
cd dojo-digital
```

> **O IP público muda a cada stop/start.** E o endereço da API está embutido no build do frontend. Se você parar a instância depois de publicar o site, terá de rebuildar e reenviar o `dist/`. Um **Elastic IP** resolve — só lembre que ele também é cobrado (seção 07).

03 O Docker Compose

O arquivo inteiro, do jeito que vai na raiz do repositório. Na página seguinte, as cinco decisões que ele contém e o que cada uma resolve.

DOCKER-COMPOSE.YML

```yaml
services:
  api:
    build: ./backend
    container_name: dojo-api
    restart: unless-stopped
    ports:
      - '3000:3000'
    environment:
      PORT: 3000
      DATABASE_URL: postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      AWS_REGION: ${AWS_REGION}
      S3_BUCKET_ARQUIVOS: ${S3_BUCKET_ARQUIVOS}
      AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID}
      AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY}
      S3_URL_EXPIRA_EM: ${S3_URL_EXPIRA_EM:-3600}
      SEED_GOLPES: ${SEED_GOLPES:-true}
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    container_name: dojo-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}']
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s

volumes:
  pgdata:
```

03 O Docker Compose — as cinco decisões

1 · O host do banco é `db`, não `localhost`

O compose cria uma rede própria e um DNS interno em que **o nome do serviço resolve para o container**. Por isso a `DATABASE_URL` aponta para `@db:5432`. Dentro do container da API, `localhost` é a própria API — é o erro número um desta tarefa.

2 · O banco não tem `ports`

A API publica `3000:3000` porque o navegador precisa alcançá-la. O banco não publica nada: quem fala com ele já está na mesma rede. Publicar `5432` colocaria o PostgreSQL na internet, protegido só por uma senha que está num arquivo `.env`.

Para provar que ficou fechado, de fora da instância:

```
nc -z -w 3 IP_PUBLICO 5432   # tem que recusar a conexão
```

3 · Volume nomeado para os dados sobreviverem

Sem `volumes:`, os dados moram na camada gravável do container e somem no primeiro `docker compose down`. Com o volume `pgdata` montado em `/var/lib/postgresql/data`, os golpes do sensei sobrevivem. O teste é direto:

```
docker compose down
docker compose up -d
curl localhost:3000/api/golpes   # os golpes têm que continuar lá
```

4 · `healthcheck` junto com `depends_on`

`depends_on` sozinho só espera o container **iniciar**, não o banco ficar **pronto**. O PostgreSQL leva alguns segundos a mais, e a API subiria antes e quebraria. A combinação de `healthcheck` com `condition: service_healthy` faz o compose esperar o `pg_isready` responder.

5 · Nenhum segredo dentro do arquivo

Tudo que aparece como `${ASSIM}` é lido do `.env` da raiz, que está no `.gitignore`. O compose pode ser versionado à vontade; o `.env`, nunca.

> **Subindo:** `cp .env.example .env`, preencha, e `docker compose up -d --build`. Acompanhe com `docker compose logs -f api`.

04 O frontend no S3, em modo SPA

1 Build com o endereço da API

```
cd frontend
echo "VITE_API_URL=http://IP_PUBLICO_DA_EC2:3000" > .env
npm install
npm run build
```

A variável entra no **build**. Trocar o `.env` depois, sem rebuildar, não muda nada no que já está no bucket.

2 Crie o bucket e libere a leitura

Nome único, mesma região. Em **Permissions**, desmarque o **Block all public access** e aplique a policy abaixo. Desmarcar sem a policy não basta; a policy sem desmarcar também não.

BUCKET POLICY DO BUCKET DO FRONTEND

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::SEU-BUCKET-FRONTEND/*"
  }]
}
```

3 Ative o modo SPA

Em **Properties → Static website hosting**:

- **Index document:** `index.html`
- **Error document:** `index.html` — este é o truque

Uma SPA tem um arquivo só. Ao pedir `/golpes`, o S3 procura um objeto com esse nome, não acha e devolve 404. Apontando o documento de erro para o `index.html`, o S3 entrega a aplicação, e é o roteador do React que decide a rota.

4 Envie o build

```
aws s3 sync dist/ s3://SEU-BUCKET-FRONTEND --delete
```

Ou pelo console, arrastando o **conteúdo** da pasta `dist/` — não a pasta.

> **Use a URL de website, não a do objeto.** São endereços diferentes: `http://bucket.s3-website-regiao.amazonaws.com` entende os documentos de índice e de erro; `https://bucket.s3.amazonaws.com` não. E como o backend é HTTP puro, uma página servida em HTTPS teria a chamada à API bloqueada pelo navegador por **mixed content**.

05 O bucket de arquivos e o par de chaves

1 Crie o segundo bucket

Separado do frontend, mesma região, e desta vez **mantenha o Block all public access ligado**. Sem static website hosting: este bucket guarda arquivos, não serve um site.

Ele pode ficar privado porque o backend gera uma **URL pré-assinada** para cada foto a cada resposta. Quem autoriza o acesso é a assinatura, que expira em uma hora.

2 Crie o usuário IAM e o par de chaves

Em **IAM → Users → Create user**, sem acesso ao console. Anexe a policy abaixo diretamente ao usuário e depois gere o par em **Security credentials → Create access key**.

POLICY DO USUÁRIO — SÓ ESTE BUCKET, SÓ ESTAS AÇÕES

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
    "Resource": "arn:aws:s3:::SEU-BUCKET-ARQUIVOS/*"
  }]
}
```

Repare no **menor privilégio**: três ações, um bucket. Nada de `s3:*` em `"Resource": "*"`.

3 Coloque as chaves no .env da instância

.ENV — NA EC2, NUNCA NO GIT

```
AWS_REGION=us-east-1
S3_BUCKET_ARQUIVOS=seu-bucket-arquivos
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
```

Não existe nenhuma linha de código no backend que mencione chave de acesso: o SDK da AWS lê estas variáveis sozinho, pela cadeia padrão de credenciais.

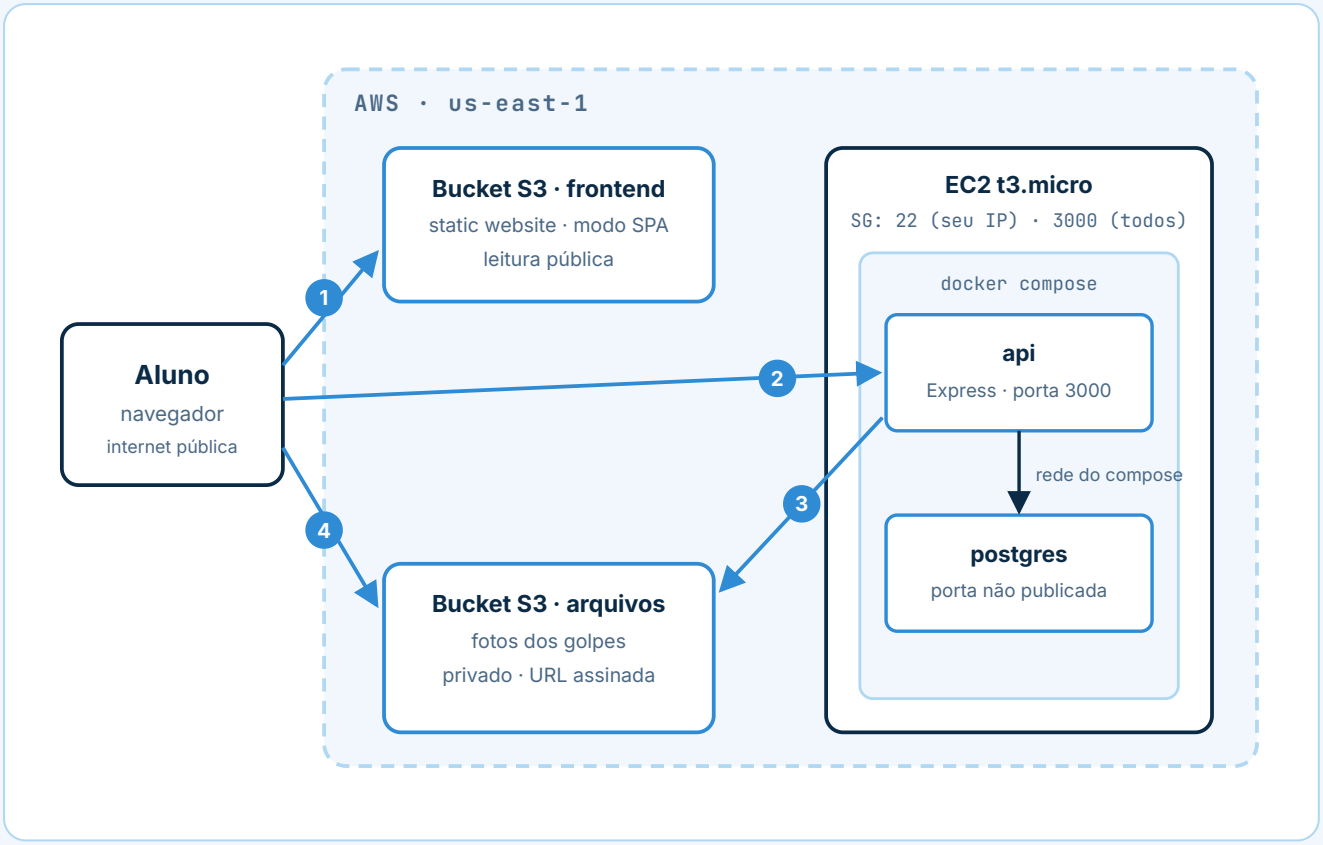
4 Reinicie e confira

```
docker compose up -d
curl localhost:3000/health
```

Tem que responder `"armazenamento": "configurado"` e `"faltando": []`. Depois cadastre um golpe com foto pelo site e veja a imagem aparecer.

> **A mesma cadeia de credenciais aceita uma IAM Role.** Anexando uma role à instância, o par de chaves some do `.env` e as credenciais passam a ser temporárias e rotacionadas pela AWS. É o que a AWS recomenda para código rodando em EC2, e cai na prova da CLF-C02. Fica como leitura.

06 O diagrama de arquitetura



#	O QUE ACONTECE
1	O navegador carrega a SPA do bucket do frontend, pela URL de website.
2	A SPA chama `http://IP:3000/api/golpes`. A API lê os metadados no PostgreSQL, que está na mesma instância e não é acessível de fora.
3	No cadastro, a API envia a foto ao bucket de arquivos e guarda no banco só a chave do objeto.
4	A API devolve uma URL pré-assinada, e o navegador baixa a imagem direto do bucket privado.

O que o diagrama precisa mostrar para valer nota: todos os componentes, o sentido de cada fluxo, a fronteira da região, o que é público e o que não é, e as portas abertas. Setas sem legenda não contam.

Repare que **toda seta atravessa a internet pública**, inclusive a de número 3, que liga dois serviços da AWS na mesma região. É exatamente esse o alvo do desafio faixa-preta, na seção 08.

07 A estimativa de custos

Montada na **AWS Pricing Calculator** (`calculator.aws`), região us-east-1. Preços conferidos na fonte oficial da AWS em **agosto de 2026**; confira de novo, porque eles mudam.

As premissas, que valem tanto quanto a conta

- 1 instância `t3.micro` ligada **24×7**, ou 730 horas por mês
- 1 volume `gp3` de 8 GB e 1 endereço IPv4 público
- 60 alunos, 80 golpes cadastrados, fotos de cerca de 400 KB
- armazenamento total menor que 2 GB, somando os dois buckets
- 60 mil requisições GET e 100 PUT por mês
- cerca de 21 GB de saída de dados para a internet por mês

Depois que o período gratuito acabar

ITEM	QUANTIDADE	PREÇO UNITÁRIO	MÊS
EC2 t3.micro	730 h	US$ 0,0104 / h	US$ 7,59
Endereço IPv4 público	730 h	US$ 0,005 / h	US$ 3,65
EBS gp3	8 GB	US$ 0,08 / GB	US$ 0,64
S3 armazenamento	2 GB	US$ 0,023 / GB	US$ 0,05
S3 requisições GET	60.000	US$ 0,0004 / mil	US$ 0,02
S3 requisições PUT	100	US$ 0,005 / mil	US$ 0,00
Saída para a internet	21 GB	100 GB/mês grátis	US$ 0,00
Total			**≈ US$ 11,95**

> **Duas surpresas nessa tabela.** A primeira: o **IPv4 público é cobrado desde fevereiro de 2024**, mesmo colado numa instância ligada, e sozinho é um terço da conta. A segunda: **armazenamento e requisições são irrelevantes** nesta escala. O que se paga aqui é uma máquina ligada o mês inteiro.

07 Custos — região e período gratuito

E se fosse em São Paulo?

ITEM	US-EAST-1	SA-EAST-1	DIFERENÇA
EC2 t3.micro por hora	US$ 0,0104	US$ 0,0168	+62%
S3 Standard por GB-mês	US$ 0,023	US$ 0,0405	+76%

São Paulo custa mais e entrega latência menor para Porto Alegre. Para um dojô com 60 alunos a latência não é problema, e a diferença de preço é real: a escolha por us-east-1 se justifica sozinha. **Justificar a região faz parte da tarefa 05** — não basta escolher.

O período gratuito, e o que acontece quando ele acaba

Aqui mora a parte que o sensei precisa entender, e que mudou recentemente. **Contas criadas a partir de 15 de julho de 2025** não têm mais o modelo antigo de 750 horas de EC2 por mês durante 12 meses. Elas recebem **US$ 100 em créditos** ao criar a conta e podem ganhar **até US$ 100 adicionais** usando serviços.

PLANO	COMO FUNCIONA
Free plan	Nada é cobrado. Termina em **6 meses ou quando os créditos acabam**, o que vier primeiro. Depois disso **a conta é encerrada** e os recursos são perdidos, com 90 dias para migrar para o plano pago.
Paid plan	Acesso a tudo. Os créditos são consumidos primeiro; o que passar é cobrado no cartão. A conta não fecha quando os créditos terminam.

Aplicando ao nosso caso: a US$ 11,95 por mês, os US$ 100 iniciais cobrem cerca de **oito meses** desta arquitetura. Ou seja, **o que vai acabar primeiro não são os créditos, é o prazo de seis meses** do plano gratuito. Contas criadas antes de 15/07/2025 seguem no modelo antigo, com 750 horas mensais de `t2.micro` ou `t3.micro` por 12 meses.

> **É isso que precisa estar escrito no relatório do sensei**, sem jargão: hoje não se paga nada, daqui a alguns meses passa a custar cerca de US$ 12 por mês, e a maior parte disso é a máquina ligada dia e noite.

08 O desafio faixa-preta

O alvo é a seta 3 do diagrama: fazer a EC2 falar com o bucket **sem sair para a internet**. A resposta é um **Gateway VPC Endpoint** para o S3 — que, ao contrário do endpoint do tipo Interface, **não custa nada**.

1 Crie o endpoint

VPC → Endpoints → Create endpoint, categoria **AWS services**, serviço `com.amazonaws.us-east-1.s3`, tipo **Gateway**. Marque a route table da subnet onde a instância está.

A AWS insere sozinha uma rota com destino a uma prefix list do S3 apontando para o endpoint. Todo tráfego da subnet para o S3 passa a sair por ali, dentro da rede da AWS.

2 Feche a porta antiga

Restrinja o bucket a aceitar só o caminho novo:

BUCKET POLICY DO BUCKET DE ARQUIVOS

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": "arn:aws:s3:::SEU-BUCKET-ARQUIVOS/*",
  "Condition": {
    "StringNotEquals": { "aws:SourceVpce": "vpce-0abc123..." }
  }
}
```

3 Prove que funciona

Remova a rota para o Internet Gateway ou feche a saída no Security Group e cadastre um golpe com foto. Se o upload continuar funcionando, o tráfego está indo pela rede interna.

> **No diagrama, a seta 3 deixa de atravessar a internet** e passa a ser uma linha interna à VPC. É essa a diferença que a entrega precisa mostrar, junto com a explicação do que muda no Data Transfer da estimativa.

09 Encerrando

A limpeza, na ordem certa

- **Termine a instância** — *terminate*, não apenas *stop*. Uma instância parada não cobra horas, mas o disco e o IP continuam contando.
- **Libere o Elastic IP**, se você criou um. Um IP alocado e sem uso é cobrado igual.
- **Esvazie e apague os dois buckets.** Bucket com objeto dentro não é apagado.
- **Apague o usuário IAM** e revogue o par de chaves de acesso.
- **Confira o Billing** no dia seguinte, para ver se algo ficou de pé.

Onde os grupos mais travam

SINTOMA	CAUSA QUASE SEMPRE
A API responde por `curl` dentro da instância, mas não de fora	Security Group sem a regra da porta 3000
O site abre, mas o catálogo fica vazio	O `VITE_API_URL` ficou apontando para `localhost` no build
`/golpes` devolve um XML do S3	Falta o *error document* apontando para o `index.html`
A API não conecta no banco	O host da `DATABASE_URL` está como `localhost` em vez de `db`
Tudo funcionava e parou depois de um stop/start	O IP público mudou; é preciso rebuildar o frontend
Erro de CORS no console do navegador	A API sobe com `cors()` aberto, então o problema quase nunca é o CORS: é a API não estar respondendo
`Access Denied` no upload	A policy do usuário IAM não cobre esse bucket, ou o nome do bucket está errado no `.env`

O que separa o grupo que tira nota cheia

Não é a plataforma no ar: isso todo mundo consegue. É saber **justificar cada regra do Security Group**, explicar por que o banco não tem `ports`, mostrar um diagrama que alguém de fora entende, e escrever a estimativa de custos numa linguagem que o sensei leia sem precisar perguntar o que é uma instância.

> **Fontes dos preços:** tabela de preços sob demanda do Amazon EC2 e Price List API do Amazon S3 (agosto de 2026); anúncio da cobrança de IPv4 público, em vigor desde 1º de fevereiro de 2024; documentação de planos do AWS Free Tier; e o anúncio da franquia de 100 GB mensais de saída de dados.